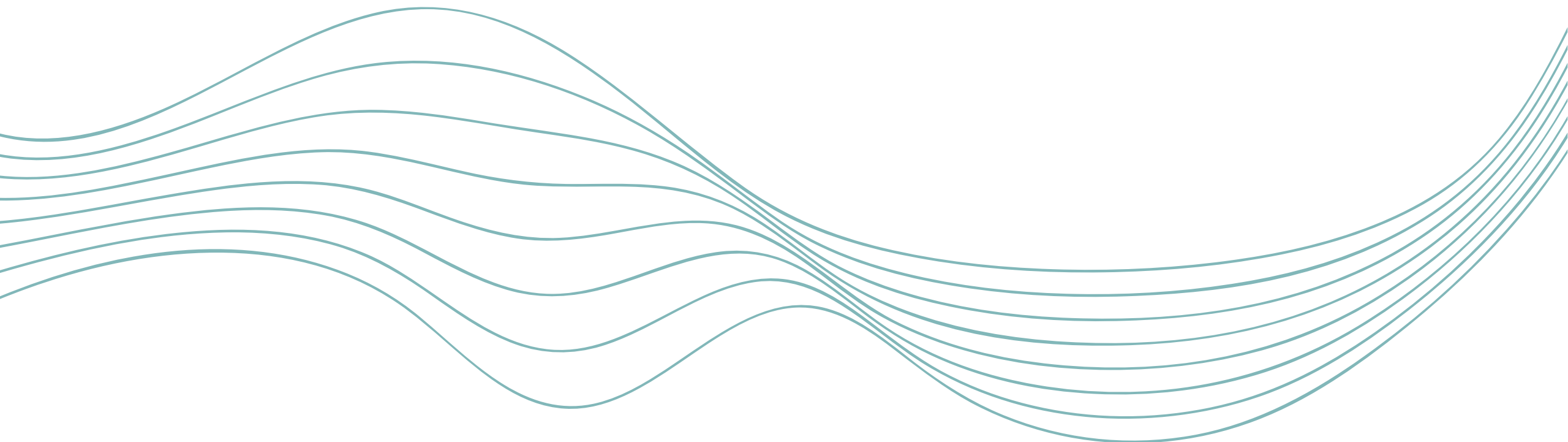
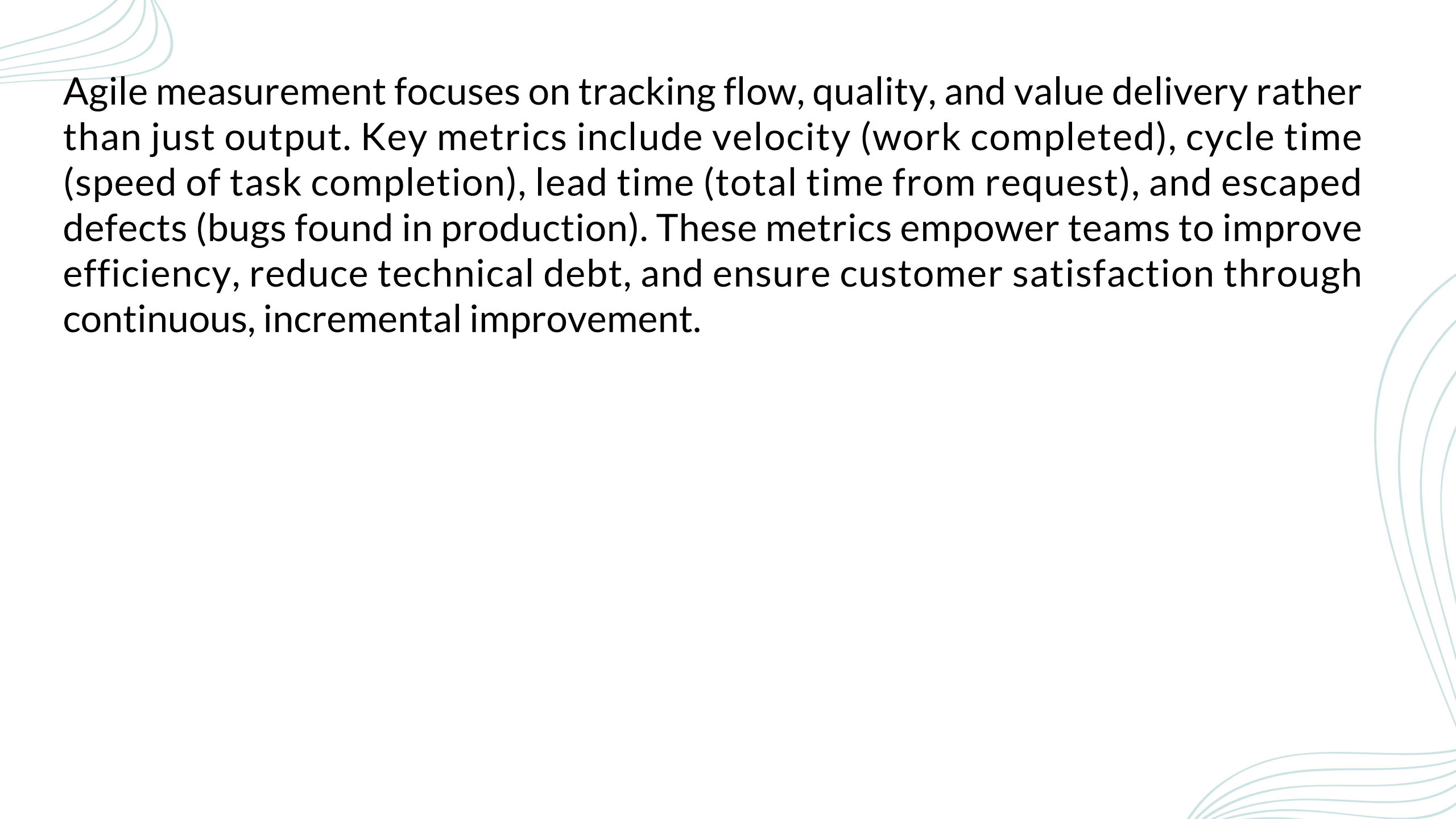


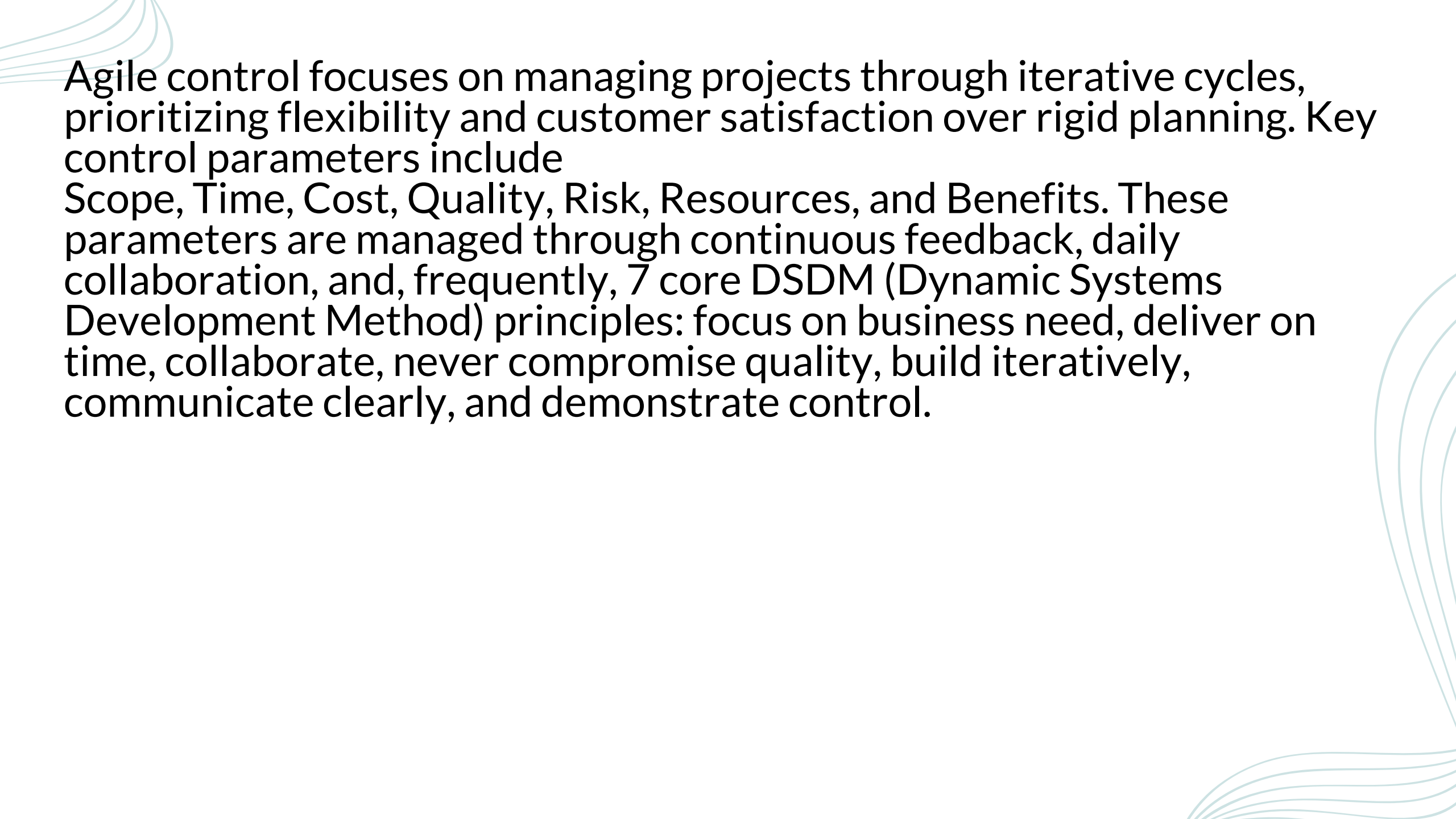


# *Unit-V*

Dr. Raghunath Dey



Agile measurement focuses on tracking flow, quality, and value delivery rather than just output. Key metrics include velocity (work completed), cycle time (speed of task completion), lead time (total time from request), and escaped defects (bugs found in production). These metrics empower teams to improve efficiency, reduce technical debt, and ensure customer satisfaction through continuous, incremental improvement.



Agile control focuses on managing projects through iterative cycles, prioritizing flexibility and customer satisfaction over rigid planning. Key control parameters include Scope, Time, Cost, Quality, Risk, Resources, and Benefits. These parameters are managed through continuous feedback, daily collaboration, and, frequently, 7 core DSDM (Dynamic Systems Development Method) principles: focus on business need, deliver on time, collaborate, never compromise quality, build iteratively, communicate clearly, and demonstrate control.

Agile controls differ from traditional project management by being flexible, but still manage essential constraints:

### **Key Agile Control Parameters**

1. **Scope (Flexible/Variable):** Unlike traditional methods, Agile accepts changing requirements late in development. The scope is not fixed; instead, it is managed through a prioritized Product Backlog.
2. **Time (Fixed):** Agile uses time-boxed iterations (Sprints), usually 1-4 weeks, to ensure regular delivery. The deadline for a sprint is rarely moved.
3. **Cost (Variable/Fixed):** While the team size and iteration length are fixed (setting a cost per sprint), the final total cost may vary based on how many iterations are needed to achieve the desired value.
4. **Quality (Fixed):** Agile maintains quality through continuous testing, integration, and defining a "Done" criterion.
5. **Risk:** Identified through continuous monitoring and mitigated early by delivering working, tested software in small increments.
6. **Resources:** Teams are typically self-organizing and cross-functional. Control involves maintaining team stability and ensuring a sustainable pace.
7. **Benefits (Value):** The main focus is delivering the highest value (benefits) to the customer first, verified via regular, early demonstrations.

# Key Control Metrics and Mechanisms

To monitor these parameters, Agile teams use:

Sprint Burndown Charts: Tracks remaining work within a sprint.

Velocity: Measures the amount of work completed in a sprint to forecast future capacity.

Daily Stand-ups: Promotes daily communication.

Retrospectives: Focuses on improving processes.

Kanban Boards: Visualizes workflow and limits work in progress (WIP).

## **An Agile approach to risk is**

proactive, continuous, and integrated into daily workflows, rather than a one-time upfront activity. By using short, iterative sprints, teams identify, assess, and mitigate risks early—often using RAID logs (Risks, Assumptions, Issues, Dependencies)—to reduce uncertainty and prevent small issues from becoming project-killers.

Key aspects of the Agile risk approach include:

- Continuous Identification: Risks are reviewed during regular meetings, such as daily stand-ups and sprint planning, not just at the project start.

- Reduced Impact: By building in small increments, the maximum risk exposure is limited to the length of a single sprint.

- Transparency: Agile boards and visual tools are used to track risks, ensuring the entire team can see and discuss potential problems.

- Proactive Mitigation: Using "spikes" (short research tasks) to address technical uncertainties early.

- The ROAM Technique: Frequently, teams categorize risks as Resolved, Owned, Accepted, or Mitigated.

The Agile approach relies heavily on constant feedback from stakeholders to pivot quickly when risks materialize, ensuring that working software remains the priority.

Configuration management (CM) involves systematically managing changes to a project's assets, such as code, documentation, and hardware components. In traditional waterfall projects, CM often emphasizes rigidity to ensure traceability and compliance. Agile projects, however, demand a balance between adaptability and maintaining control.



In satellite projects, where precision and compliance are non-negotiable, CM helps track even the smallest changes to critical components. Similarly, in software projects, CM ensures that the fast-paced iteration cycles of Agile don't lead to chaos or quality issues.

Agile methodologies prioritize responsiveness and minimal overhead. This can seem at odds with CM, which requires meticulous tracking and documentation.

## Common challenges include:

**Frequent Iterations and Updates:** Agile teams deliver updates in sprints or iterations. Without proper CM, these rapid changes can lead to version control issues or rework.

**Cross-functional Collaboration:** Agile emphasizes collaboration between development, operations, and business teams. Ensuring all stakeholders work with consistent and updated configurations is essential.

**Balancing Documentation with Agility:** Agile's preference for lightweight documentation often conflicts with the detailed records CM traditionally requires.

Successfully integrating CM into Agile projects requires tailoring traditional practices to fit Agile's fast-paced, iterative nature.

## Here are some best practices :

**Adopt Robust Version Control Systems:** Tools like Git or Mercurial( I don't use) are indispensable in Agile software development. They allow teams to track changes, merge updates, and roll back if needed. For satellite projects, CM tools must handle both hardware and software components seamlessly.

**Automate Where Possible:** Automation reduces the overhead of manual tracking. Continuous integration (CI) and continuous deployment (CD) pipelines can automate versioning, testing, and deployment processes.

**Define Minimal Viable Processes:** Establish lightweight CM processes that ensure traceability without stifling agility. For instance, enforce tagging of major releases and automated audit trails for changes.

**Emphasize Communication and Training:** Ensure that all team members understand the CM process and its tools. Regular cross-functional reviews can ensure alignment across teams.

**Leverage Agile CM Tools:** Modern tools like JIRA and Azure DevOps offer built-in support for configuration tracking, making them ideal for Agile environments.

## CONFIGURATION MANAGEMENT TOOLS



**Terraform**

**'SALTSTACK**



**puppet**



**git**



**CHEF**



ANSIBLE



**docker**



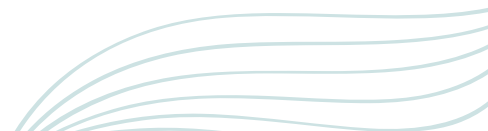
**DSDM Atern (now commonly referred to just as DSDM)** is an Agile framework focused on delivering business value through eight core principles. These principles emphasize on-time delivery, collaboration, quality, and iterative development, ensuring that projects align with strategic goals, use MoSCoW prioritization, and build upon solid foundations. The 8 DSDM Atern Principles-

Focus on the business need: All project decisions must align with business priorities, ensuring the delivery of the Minimum Usable Subset (MUS) of features.

Deliver on time: Consistently delivering products on time is paramount. Timeboxing and prioritizing requirements are key to meeting deadlines.

Collaborate: Atern relies on team collaboration with active stakeholder involvement to foster trust and shared ownership.

Never compromise quality: The quality criteria are defined at the start. The project must meet these agreed-upon standards, as quality cannot be sacrificed.



### **Build incrementally from firm foundations:**

Projects should start with a solid foundation, ensuring that high-level requirements and business cases are understood before full development.

Develop iteratively: An iterative approach (timeboxing) allows for regular feedback and refinement of the product, adapting to changes as necessary.

Communicate continuously and clearly: The framework emphasizes open, informal communication, using daily stand-ups and workshops to keep teams aligned.

Demonstrate control: Projects must demonstrate that they are under control, using clear progress tracking and reporting.

The Dynamic Systems Development technique (DSDM) is an associate degree agile code development approach that provides a framework for building and maintaining systems. The DSDM philosophy is borrowed from a modified version of the sociologist principle—80 % of An application is often delivered in twenty percent of the time it'd desire deliver the entire (100 percent) application. DSDM is An iterative code method within which every iteration follows the 80% rule that simply enough work is needed for every increment to facilitate movement to the following increment. The remaining detail is often completed later once a lot of business necessities are noted or changes are requested and accommodated.

## Summary of the Benefits of Using DSDM- Rationale for ATERN

Using iterative development, DSDM involves the solution's business stakeholders throughout the project lifecycle. This has many benefits, for example:

- The business is more likely to feel ownership of the solution as it evolves and, most importantly, as it transitions into live use

- Prioritisation will enable a project to be delivered on time whilst protecting the quality of what is being delivered

- The risk of building the wrong solution is greatly reduced

- The final solution is more likely to meet the real business need

- Deployment is more likely to go smoothly, because of the co-operation of all parties concerned throughout development

DSDM specifically addresses many of the problems which cause projects to struggle or to fail. For many organisations, having the ability to deliver working solutions consistently, on time and on budget, is seen as a major step forward. DSDM will provide this.

## **Refactoring in Agile is**

the continuous process of restructuring existing code to improve its internal structure, readability, and maintainability without changing its external behavior. It reduces technical debt, prevents code rot, and allows teams to adapt to changing requirements efficiently by keeping the codebase clean and manageable.

### **Key Aspects of Refactoring in Agile:**

**Continuous Improvement:** Unlike traditional methods, Agile encourages constant, incremental refactoring rather than a single, large overhaul. It is often done during daily development, such as with TDD (Test-Driven Development).

**Behavior-Preserving:** The goal is to clean up code (e.g., removing duplication, simplifying complex logic) while ensuring all existing automated tests remain green.

**Preventing "Throw-away" Code:** It tackles technical debt, preventing systems from becoming unmaintainable and ensuring sustainable development speed.

**Refactoring Techniques:** Common techniques include renaming variables, moving methods to better classes, extracting methods for simplicity, and replacing conditional logic with polymorphism.

**Challenges:** Key challenges include balancing refactoring time within time-boxed iterations, mitigating the risk of breaking functionality, and ensuring backward compatibility.



## How to Incorporate Refactoring:

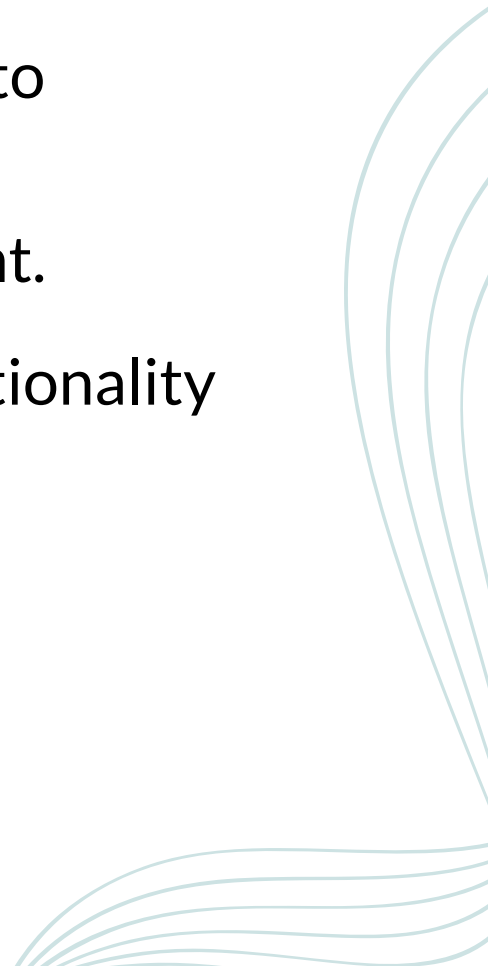
**Refactor Routinely:** Small, incremental improvements should be part of daily tasks.

**Make it Part of the Backlog:** Product Owners can prioritize refactoring tasks within the team backlog to ensure it receives attention.

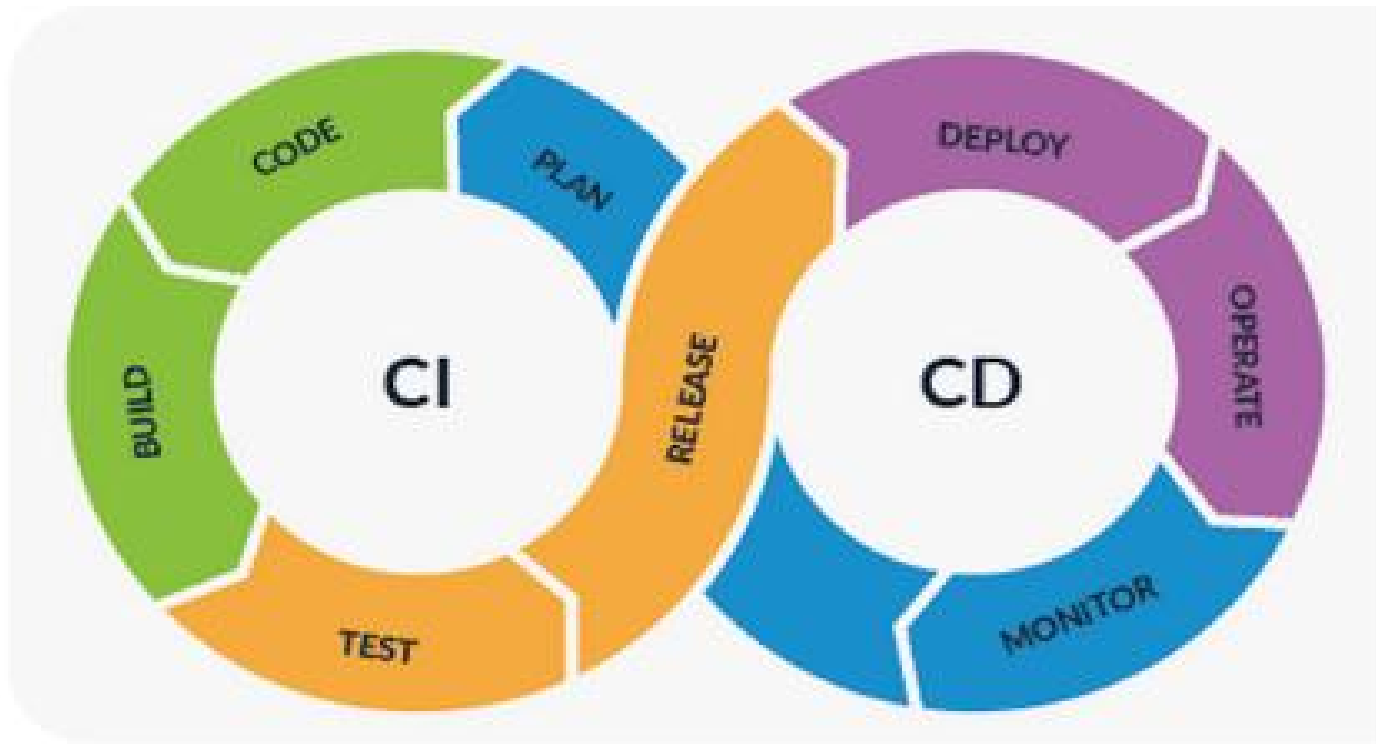
**Use Automated Tests:** Robust unit and acceptance tests are critical to ensure that refactoring does not break functionality.

**Prioritize High-Change Areas:** Focus on parts of the system that are frequently modified, as these provide the highest return on investment.

Refactoring ensures that the software structure supports future functionality rather than hindering it.



**Continuous integration (CI)** in Agile is a development practice where developers frequently commit code changes to a shared repository, triggering automated builds and tests. This process enables early bug detection, reduces integration conflicts, and ensures high-quality, stable code, directly supporting iterative, fast-paced Agile development cycles.



## Key Aspects of CI in Agile:

**Frequent Code Integration:** Developers commit code to the main branch daily (or more) to avoid merging conflicts.

**Automated Testing & Building:** Every commit triggers automated unit and integration tests to immediately validate functionality.

**Rapid Feedback:** If code breaks the build or fails tests, developers are notified immediately to fix it, maintaining a stable codebase.

**Shared Repository:** A single, central source of truth for the codebase, fostering collaboration and visibility.

**Common Tools:** Popular tools for automating the pipeline include Jenkins, GitLab CI/CD, CircleCI, and TeamCity.



## Benefits to Agile Teams:

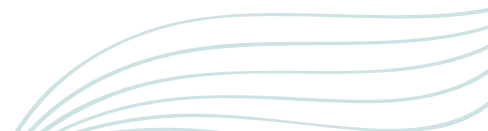
**Early Bug Detection:** Automated testing catches issues early, lowering the cost and effort of fixing them.

**Reduced Risk:** Frequent, smaller integrations minimize, or eliminate, complex "merge hell" situations.

**Faster Development Cycles:** Automated builds speed up the workflow from development to deployment.

**Increased Productivity:** Developers spend less time on manual integration and more on delivering new features.

CI is essentially the technical backbone of the agility that CI/CD, DevOps, and agile methodologies promise, acting as a crucial enabler for teams that want to deliver working software quickly and efficiently.



## **Scaling Agile for large projects involves**

applying Agile principles across multiple teams, using frameworks like SAFe, LeSS, or Scrum at Scale to manage complex dependencies and coordinate efforts. Key strategies include maintaining synchronized cadences, cross-functional teams, and utilizing "scrum of scrums" for alignment.

## **Key Frameworks for Scaling Agile**

**Scaled Agile Framework (SAFe):** A comprehensive, multi-level framework best suited for large enterprises (50-1000+ members) requiring high structure and coordination.

**Large-Scale Scrum (LeSS):** Applies Scrum principles to large-scale development by keeping structures simple and reducing dependencies.

**Scrum at Scale (S@S):** A, scalable, pyramid-like structure that extends Scrum to the organizational level.

**Disciplined Agile (DA) 2.0:** Offers a flexible approach, focusing on decision-making and choosing the right way of working.

## Core Strategies for Success

**Synchronized Cadence:** Align all teams to the same iteration length (e.g., 2-4 weeks) and release schedule.

**Cross-Functional Teams:** Structure teams to be self-sufficient, with all necessary skills to deliver completed work increments.

**Scrum of Scrums:** Implement a, "scrum of scrums" approach for regular, high-level, inter-team planning and dependency management.

**Prioritized Backlog:** Create a master, top-level product backlog to manage dependencies and align efforts across the enterprise.

**Focus on Culture:** Shift from top-down management to a, coaching, leadership style that empowers teams and fosters transparency.

## Common Scaling Challenges

**Culture Shift:** Resisting the change from traditional management to agile methodologies.

**Complex Dependencies:** Managing, dependencies across numerous teams and departments.

**Increased Coordination:** Ensuring, effective communication and alignment across all teams.

For organizations just starting, Scrum at Scale offers a simpler path by, building directly on the familiar Scrum methodology. This YouTube video provides a visual overview of scaling agile in large organizations.



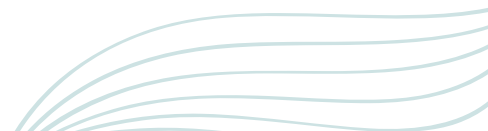
## What is Scrum of Scrums?

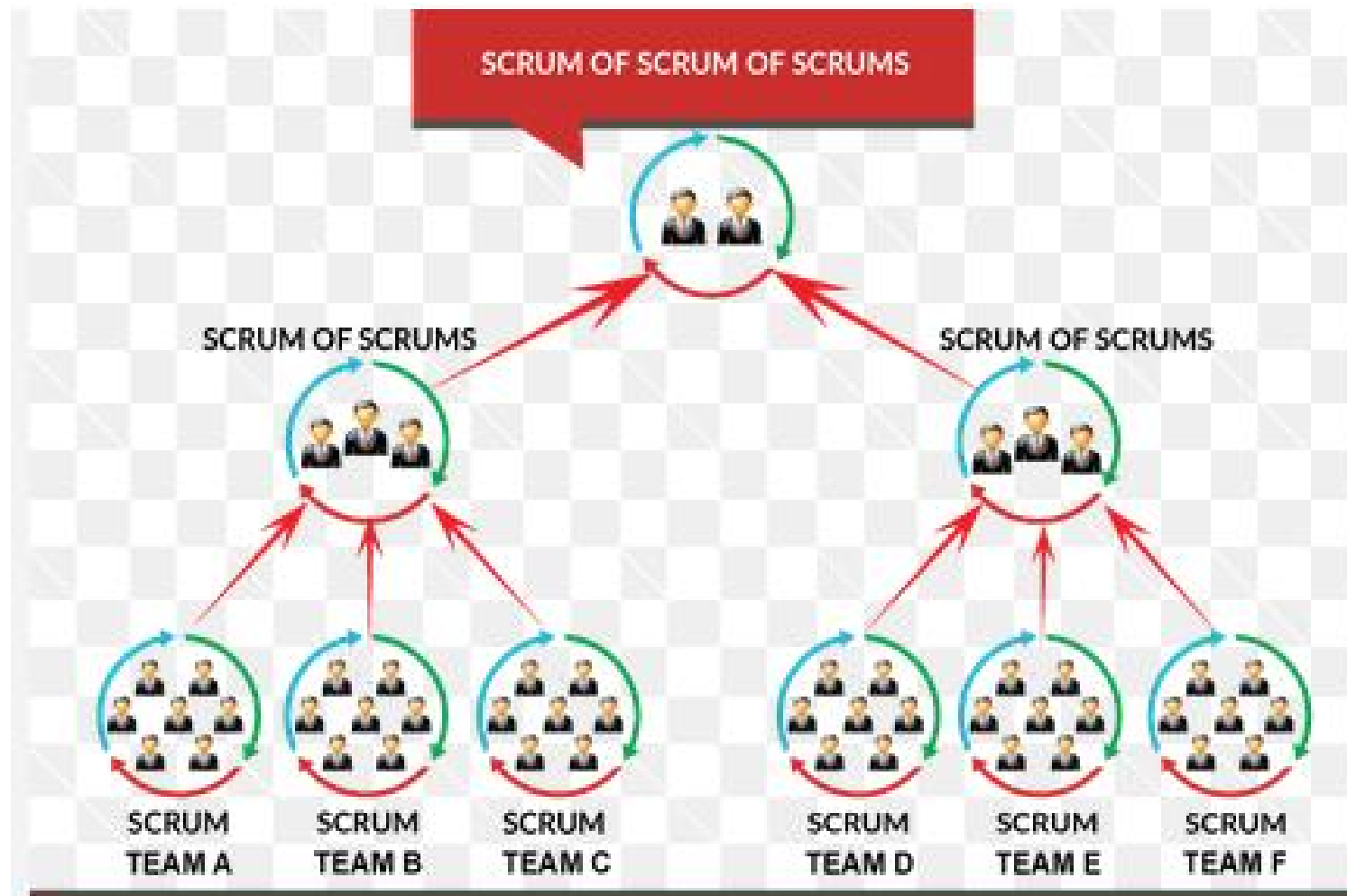
A technique to scale Scrum up to large groups (over a dozen people), consisting of dividing the groups into Agile teams of 5-10. Each daily scrum within a sub-team ends by designating one member as an “ambassador” to participate in a daily meeting with ambassadors from other teams, called the Scrum of Scrums.

Depending on the context, ambassadors may be technical contributors, each team’s Scrum Master, or even managers of each team.

The Scrum of Scrums proceeds otherwise as a normal daily meeting, with ambassadors reporting completions, next steps, and impediments on behalf of the teams they represent. Resolution of impediments is expected to focus on the challenges of coordination between the teams; solutions may entail agreeing to interfaces between teams, negotiating responsibility boundaries, etc.

The Scrum of Scrum will track these items via a backlog of its own, where each item contributes to improving between-team coordination.





## **Agile Teams - Scaled Agile Framework**

Team collaboration in scaling agile involves aligning multiple cross-functional teams toward common goals, breaking down silos, and maintaining high-performance through shared, transparent communication. It relies on structured events like Product Increment (PI) planning, Communities of Practice, and shared tools to manage interdependencies, fostering autonomous yet synchronized team behaviors.

### **Key strategies for effective collaboration in scaled agile environments include:**

**Structured Cross-Team Planning:** Utilizing ceremonies like PI Planning (as in SAFe) allows teams to align on goals, dependencies, and risks regularly.

**Communities of Practice (CoPs):** Establishing guilds (e.g., DevOps, Frontend) encourages knowledge sharing across functional areas and team boundaries.

**Product Owner Collaboration:** When multiple teams work on one product, forming a Product Owner team or using a Chief Product Owner helps maintain a unified product backlog and direction.

**Transparent Communication Channels:** Utilizing shared, automated tools (e.g., Jira, Slack) provides real-time visibility into dependencies, reducing communication lags.

**Joint Retrospectives:** Beyond individual team retrospectives, teams should engage in joint Inspect and Adapt events to identify improvements that benefit the entire organization.

**Fostering a Culture of Trust:** Encouraging a collaborative environment where teams feel safe sharing ideas and providing feedback is essential for maintaining agility at scale.

## Scrum project estimation

uses relative sizing—such as story points or T-shirt sizes—rather than exact hours to forecast effort, complexity, and risk. Teams often use techniques like Planning Poker (using Fibonacci sequences: 1, 2, 3, 5, 8...) during backlog refinement to reach consensus, allowing for faster, more accurate forecasting of sprint capacity and release schedules.

### Key Scrum Estimation Techniques-

**Planning Poker:** The most common technique, where developers privately rate stories using Fibonacci cards, then discuss discrepancies until consensus is reached.

**Relative Sizing/Estimation:** Comparing new user stories to previously completed items of known size to estimate effort rather than using hours.

**T-Shirt Sizes:** Categorizing work into sizes like XS, S, M, L, or XL to represent complexity.

**Dot Voting:** Used for quick prioritization in backlog grooming.

## Tracking in SCRUM-

Here are 9 metrics that teams can track to evaluate performance and set their own KPIs. Product teams should decide which metrics to track, how often to track them, and how to communicate them with the larger team.

### 1. Story Completion Ratio

The story completion rate is the ratio of actual stories completed to the number of stories the team committed to. Product teams commit to stories during sprint planning and review the number of stories completed during the sprint review. If a team commits to 10 stories during sprint planning and delivers 8 by sprint review, their story completion ratio is 80%.

Story Completion Ratio = # of Stories Delivered / # Number of Stores Committed

### 2. Technical Debt Index

In software product development, technical debt refers to the work that builds up when developers implement quick, short-term solutions instead of working through better, more labor-intensive solutions. Technical debt is the cost of slapping a band-aid on a product to make a quick fix or complete an MVP.

Technical debt can be measured using a debt index. You can calculate the debt index by dividing the total technical debt by another metric that represents the full scale of the project or system. For example, you can calculate the debt index by dividing the total technical debt (estimated hours of work or story points) by the codebase size (measured by lines of code). The resulting ratio approximates a scaled picture of the remaining technical debt.



Technical Debt Index = Technical Debt / Codebase Size

### 3. Velocity

Velocity is one of the most important metrics in product management. Velocity represents the consistency of the teams' estimates over time.

Actual velocity is the sum of units (often story points) delivered in a sprint and is calculated as the number of delivered units divided by the total number of sprints.

For example, if a team delivers 15 story points in 1 sprint, their actual velocity is 15.

Actual Velocity = 15 story points / 1 sprint = 15

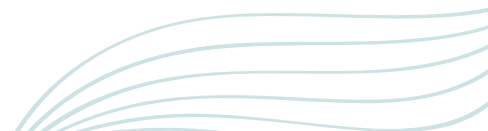
If a team delivers 100 story points in 4 sprints, their actual velocity is 25.

Actual Velocity = 100 story points / 4 sprints = 25

Expected velocity is the total estimated story points divided by the total number of planned sprints.

For example, if a team estimates they will complete 45 story points in 3 sprints, their expected velocity is 15.

Expected Velocity = 45 story points / 3 sprints = 15



Actual velocity isn't a measure of productivity; it simply represents an average unit of completion based on past data. Actual velocity may change due to outside factors like holidays, team changes, and technical issues. For this reason, teams should use actual velocity as a planning tool. Over time, actual velocity can be compared to estimated velocity to see how team output differs against expectations.

#### 4. Burndown

Sprint burndown helps teams measure the work remaining during the sprint's execution. Sprint burndown is a visual representation, usually a line chart over time, that represents the rate at which the team completes work compared to how much work still needs to be done. On a burndown chart, the x-axis represents time increments, while the y-axis represents the number of tasks yet to be completed.

A burndown graph representing the decline of outstanding story points over time.

#### 5. Escaped Defects

The number of escaped defects helps product teams evaluate the quality of their product. This metric shows how many bugs, or defects, "escaped" testing and were experienced by users in production. In an ideal world, scrum teams can thoroughly test stories to avoid escaped defects. In reality, sometimes bugs get deployed to production.

Defect density, which measures the number of defects per software size, such as lines of code, can also help teams understand their product quality.

Tracking the number of escaped defects and defect density over time can help your team understand the quality of releases and whether that quality improves or degrades over time.



## 6. Sprint Goal Success

Sprint goals are one—to two-sentence statements that outline the goals of a sprint. They are an optional part of the scrum framework, but tracking this metric over time provides insight into whether teams meet their goals. If a team consistently misses goals, this data can help them investigate whether their goals are realistic. Furthermore, it can help them identify roadblocks to goal success.

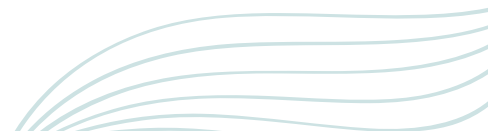
## 7. Workload Distribution

A strong product team is essential to creating a strong product. Monitoring workload distribution helps ensure that all team members take on what they can handle. Some team members may be overworked without visibility into workload distribution, while others may be underutilized. Ultimately, both issues can contribute to low employee satisfaction.

Product teams may use workload distribution as a proxy variable for productivity, which can create stress for employees. When tracking workload over time, product teams should take care to create an environment of trust and psychological safety.

## 8. Team Satisfaction

Team satisfaction is a significant component of a successful scrum team. If your team members don't feel that their needs are met and aren't enthusiastic about their work, then no process or methodology can fix your team's problems. Checking in on workload, work/life balance, happiness, and motivation helps you gather data on your team's satisfaction and shows that you care about your team as people.



## 9. Customer Satisfaction (NPS)

While there are several ways to measure customer satisfaction, Net Promoter Score (NPS) is among the most common. NPS measures whether users would recommend the product to others. Strong, consistent NPS scores mean the product team delivers valuable products to customers.

### Conclusion

Monitoring and tracking scrum metrics can support agile product management in several ways. By recording and visualizing this data, scrum teams can enhance their efficiency, effectiveness, and ability to adapt quickly to changing needs or priorities. Tracking metrics also helps ensure product teams are satisfied and connected to the organization's broader customer service goals.

## Effective communication in Scrum projects

It is built on transparency, inspection, and adaptation, facilitated through regular ceremonies like Daily Stand-ups, Sprint Planning, Reviews, and Retrospectives. It ensures alignment on goals, encourages immediate feedback, and fosters collaboration between the Product Owner, Scrum Master, and Development Team to remove impediments and deliver value.

### Key Communication Components in Scrum:

Daily Stand-up/Scrum (15 mins): Team sync-point to discuss progress, goals, and blockers.

Sprint Planning: Defines the what and how of the sprint, aligning on a clear goal.

Sprint Review: Stakeholder feedback session on the working product increment.

Sprint Retrospective: Internal team meeting to discuss process improvements for the next sprint.

Backlog Refinement: Ongoing activity to clarify, estimate, and break down requirements.

## **Best Practices for Scrum Communication:**

**Foster Psychological Safety:** Create an environment where team members feel comfortable sharing ideas, asking questions, and providing feedback without fear of judgment.

**Emphasize Face-to-Face:** Prioritize in-person conversations or video calls for better communication, as recognized by Agile principles.

**Use Visual Management:** Utilize Kanban boards, burndown charts, and product increments for shared visibility.

**Clear Roles:** Ensure the Scrum Master facilitates, the Product Owner clarifies requirements, and the Development Team communicates technical progress.

**Active Listening:** Utilize techniques like paraphrasing to ensure shared understanding and reduce noise in communication Medium.

## **Role of the Scrum Master in Communication:**

The Scrum Master fosters communication by modeling behaviors, encouraging interaction, and ensuring a safe space for open dialogue. If a team is not talking, the Scrum Master should prompt discussions and use, for example, smaller breakout groups to boost participation.

## **Best practices for managing Scrum projects include**

empowering cross-functional teams, maintaining a prioritized product backlog, holding efficient, time-boxed ceremonies (daily standups, retrospectives), and using visual tools like Scrum boards to track progress. Prioritize transparency to ensure everyone is aligned with project goals and can identify bottlenecks early.

### **Key Scrum Best Practices:**

**Empower the Team & Foster Collaboration:** Encourage self-management within the development team and maintain close, consistent communication.

**Prioritize the Product Backlog:** The Product Owner should keep the backlog ordered by value, ensuring the highest priority items are addressed first.

**Define Clear "Done" Criteria:** Establish a shared, strict Definition of Done (DoD) to ensure quality and completeness of user stories.

**Use Visual Management (Scrum Boards):** Utilize digital or physical boards to map tasks, providing real-time visibility into progress and potential bottlenecks.

**Strict Time-Boxing:** Keep daily standups, sprint planning, reviews, and retrospectives focused and within their designated time limits.



**Run Productive Retrospectives:** Use the retrospective to focus on actionable improvements rather than just complaining about issues.

**Measure Velocity & Burndown:** Use sprint burndown charts and velocity tracking to monitor progress and improve forecasting, as explained in this KnowledgeHut article.

**Protect the Sprint:** Prevent the team from adding new work after the sprint has begun, as described in this Microsoft Learn article.

**Adopt an Agile Mindset:** Focus on delivering value and adapting to change over strictly adhering to a rigid, upfront plan.

**Scale Wisely (Scrum of Scrums):** When multiple teams work on one product, use "Scrum of Scrums" meetings to coordinate and synchronize sprints, as discussed in this Techademy article.

For managing multiple teams, ensuring they all work from a single, unified backlog is crucial to maintain consistency

